Enumerations, Structures and Classes:

1.

```
enum BookStatus {
    case available
    case borrowed(by: String, dueIns: Int)
    case reserved(for: String)
  var label: String {
        switch self {
        case .available:
            return "Available"
        case .borrowed(let name, let days):
            return "Borrowed by \(name), due in \(days) days"
        case .reserved(let name):
            return "Reserved for \(name)"
        }
    }
}struct Book {
    var title: String
    var status: BookStatus = .available
}class Member {
    var name: String
    var borrowedBooks: [String]
init(name: String, borrowedBooks: [String] = []) {
        self.name = name
        self.borrowedBooks = borrowedBooks
    }
}
```

```
var book1 = Book(title: "Swift Programming")

var book2 = book1

book2.status = .borrowed(by: "Ravi", dueIns: 5)

print("Book 1: \(book1.title) - \(book1.status.label)")

print("Book 2: \(book2.title) - \(book2.status.label)")

let member1 = Member(name: "Arun")

let member2 = member1

member2.name = "Kumar"

print("Member 1: \(member1.name)")

print("Member 2: \(member2.name)")
```

Output

```
Book 1: Swift Programming - Available
Book 2: Swift Programming - Borrowed by Ravi, due in 5 days
Member 1: Kumar
Member 2: Kumar
```

2.

```
import Foundation

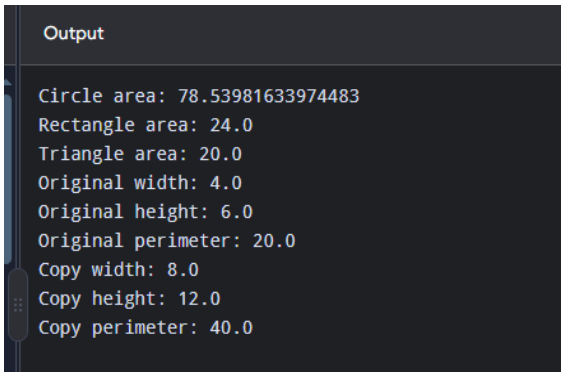
enum Shape {
   case circle(radius: Double)
   case rectangle(width: Double, height: Double)
   case triangle(base: Double, height: Double)
var area: Double {
      switch self {
      case .circle(let radius):
         return Double.pi * radius * radius
      case .rectangle(let width, let height):
         return width * height
      case .triangle(let base, let height):
```

```
            return 0.5 * base * height
        }
    }
}
struct Rectangle {
    var width: Double
    var height: Double
var perimeter: Double {
        return 2 * (width + height)
    }
mutating func scale(by factor: Double) {
        width *= factor
        height *= factor
    }
}
let circle = Shape.circle(radius: 5)
let rectangleShape = Shape.rectangle(width: 4, height: 6)
let triangle = Shape.triangle(base: 8, height: 5)
print("Circle area: \(circle.area)")
print("Rectangle area: \(rectangleShape.area)")
print("Triangle area: \(triangle.area)")
var rect1 = Rectangle(width: 4, height: 6)
var rect2 = rect1
rect2.scale(by: 2)
print("Original width: \(rect1.width)")
print("Original height: \(rect1.height)")
print("Original perimeter: \(rect1.perimeter)")
```

```
print("Copy width: \(rect2.width)")

print("Copy height: \(rect2.height)")

print("Copy perimeter: \(rect2.perimeter)")
```



3.

```
struct Counter {
    var value = 0


    mutating func increment() {
        value += 1
    }
}


class Box {
    var value = 0


    func increment() {
        value += 1
    }
}


var c = Counter()
```

```
c.increment()

print("Counter value: \(c.value)")


let b = Box()

b.increment()

print("Box value: \(b.value)")


let b2 = b

print("Same object: \(b === b2)")
```

